C++ Strings and Character Arrays - Questions and Solutions

Complete collection of string manipulation questions with detailed explanations

---

Question 1: Constant Member Functions

Which constant member functions does not modify the string?

- A) empty()
- B) assign()
- C) append()
- D) erase()

Answer: A) empty()

Explanation: Constant member functions are functions that do not modify the object they are called on. Among the options:

- ✅ `empty()` - Checks if string is empty, doesn't modify it (const function)
- ❌ `assign()` - Assigns new content to string (modifies)
- ❌ `append()` - Appends text to string (modifies)
- ❌ `erase()` - Erases characters from string (modifies)

Question 2: Character Array Access

What will be the output of the following code?

```cpp
#include <iostream>
#include <string>
using namespace std;
int main(){
   char str[] = "Hello World";
   cout << str[0];
   return 0;
}
```

- A) H
- B) e

- C) Error
- D) 0

Answer: A) H

Explanation: `str` is a C-style character array initialized with "Hello World". Arrays are 0-indexed in C/C++, so `str[0]` is the first character, which is 'H'.

Question 3: String Input with Spaces

What will be the output of the following C++ code if the string entered by the user is "Hello World"?

```cpp
#include <iostream>
#include <string>
using namespace std;
int main(){
    string str;
    cin >> str;
    cout << str;
    return 0;
}
```

- A) Hello World
- B) Hello
- C) World
- D) Error

Answer: B) Hello

Explanation: `cin >> str` reads only the first word before the first whitespace. So `str` gets "Hello" and "World" is ignored. To read the entire line including spaces, use `getline(cin, str)`.

Question 4: String Concatenation Error

What will be the output of the following C++ code?

```cpp
```

```cpp
#include <iostream>
#include <string>
using namesapce std;  // Note the typo
int main(){
    char s1[6] = "Hello";
    char s2[6] = "World";
    char s3[12] = s1 + " " + s2;  // Invalid operation
    cout << s3;
    return 0;
}
```

- A) Hello World
- B) Hello
- C) World
- D) Error

Answer: D) Error

Explanation: Two compilation errors:

1. Typo: `using namesapce std;` should be `using namespace std;`
2. Invalid concatenation: `char s3[12] = s1 + " " + s2;` - You cannot concatenate C-style strings using `+` operator during initialization.

Question 5: String Concatenation Methods

Which of the following is correct way of concatenating two string objects in C++?

Way 1:

```cpp
string s1 = "hello";
string s2 = "world";
string s3 = s1 + s2;
```

Way 2:

```cpp
```

```
string s1 = "hello";
string s2 = "world";
string s3 = s1.append(s2);
```

Way 3:

```
cpp

string s1 = "hello";
string s2 = "world";
string s3 = strcat(s1, s2);
```

- A) 1 and 2
- B) 2 and 3
- C) 1 and 3
- D) 1, 2 and 3

Answer: A) 1 and 2

Explanation:

- ✅ Way 1: `std::string` supports `+` operator for concatenation
- ✅ Way 2: `append()` is a member function that concatenates strings
- ❌ Way 3: `strcat()` works only with C-style strings (char[]), not `std::string`

Question 6: String Length Functions

Which function is used to get the length of a string object?

- A) length()
- B) size()
- C) max_size()
- D) both size() and length()

Answer: D) both size() and length()

Explanation: In C++'s `std::string` class, both `size()` and `length()` return the same value - the number of characters in the string. `max_size()` returns the maximum possible size the string can hold, not the current length.

Question 7: Null Terminator in Strings

What will be the output of the following C++ code?

```cpp
#include <iostream>
#include <string>
#include <cstring>
using namespace std;
int main(int argc, char const *argv[])
{
    const char *a = "Hello\0World";
    cout << a;
    return 0;
}
```

- A) Hello World
- B) Hello
- C) World
- D) Error

Answer: B) Hello

Explanation: The string literal contains `\0` (null terminator) after "Hello". In C/C++, `\0` marks the end of a string, so `cout << a` stops printing at the first `\0` and only prints "Hello".

Question 8: Character Appending Methods

Which is the correct way of concatenating a character at the end of a string object?

Way 1:

```cpp
string s;
s = s + a;
```

Way 2:

```cpp
```

```
string s;
s.push_back('a');
```

- A) 1 only
- B) 2 only
- C) both of them
- D) both are wrong

Answer: C) both of them

Explanation:

- ✅ Way 1: `std::string` supports concatenating with characters using `+` operator
- ✅ Way 2: `push_back()` directly appends a single character to the end of the string (more efficient for single characters)

Question 9: String back() Function

What will be the output of the following C++ code?

```cpp
#include <iostream>
#include <string>
using namespace std;
int main ()
{
  string str ("Sanboundry.");
  str.back() = '!';
  cout << str << endl;
  return 0;
}
```

- A) Sanboundry.!
- B) Sanboundry.
- C) Sanboundry!
- D) Sanboundry!.

Answer: C) Sanboundry!

Explanation: `str.back()` returns a reference to the last character of the string. The line `str.back() = '!'` replaces the last character '.' with '!', resulting in "Sanboundry!".

Question 10: String substr() Function

What will be the output of the following C++ code?

cpp

```cpp
#include <iostream>
#include <string>
using namespace std;
int main ()
{
   string str ("sanboundry.");
   cout << str.substr(3).substr(4) << endl;
   return 0;
}
```

- A) foundry.
- B) undry.
- C) dry.
- D) found

Answer: C) dry.

Explanation:

- `str.substr(3)` returns substring starting from index 3: "boundry."
- `"boundry.".substr(4)` returns substring starting from index 4: "dry."

Question 11: String Operations with C Functions

What will be the output of the following C++ code?

cpp

```cpp
#include <iostream>
#include <cstring>
using namespace std;
int main ()
{
   char str1[11] = "Hello";
   char str2[10] = "World";
   char str3[10];
   int len;

   strcpy(str3, str1);    // str3 = "Hello"
   strcat(str1, str2);    // str1 = "HelloWorld"
   len = strlen(str1);    // len = 10

   cout << len << endl;
   return 0;
}
```

- A) 5
- B) 55
- C) 11
- D) 10

Answer: D) 10

Explanation:

- `strcpy(str3, str1)` copies "Hello" to str3
- `strcat(str1, str2)` concatenates "World" to str1, making it "HelloWorld"
- `strlen(str1)` returns 10 (length of "HelloWorld")

Question 12: Reverse Iterator Compilation Error

What will be the output of the following C++ code?

```cpp
```

```cpp
#include <iostream>
#include <string>
using namespace std;
int main ()
{
   string str = "microsoft";
   string::reverse_iterator r;

   for (r = str.rbegin(); r < str.rend(); r++)
      cout << "r;  // Syntax error here

   return 0;
}
```

- A) microsoft
- B) micro
- C) tfosorcim
- D) tfos

Answer: Compilation Error

Explanation: The line `cout << "r;` has a syntax error - the string literal is not properly closed. It should be `cout << *r;` to print the reverse characters. If corrected, it would output "tfosorcim".

Question 13: String rfind() and replace()

What will be the output of the following C++ code?

```cpp
```

```cpp
#include <iostream>
#include <string>
using namespace std;
int main ()
{
  string str ("nobody does like this");
  string key ("nobody");
  size_t f;

  f = str.rfind(key);
  if (f != string::npos)
    str.replace (f, key.length(), "everybody");

  cout << str << endl;
  return 0;
}
```

- A) nobody does like this
- B) nobody
- C) everybody
- D) everybody does like this

Answer: D) everybody does like this

Explanation:

- `str.rfind(key)` finds the last occurrence of "nobody" (at index 0)
- `str.replace(f, key.length(), "everybody")` replaces "nobody" (6 characters) with "everybody"
- Result: "everybody does like this"

Question 14: String erase() Function

What will be the output of the following C++ code?

```cpp
```

```cpp
#include <iostream>
#include <string>
using namespace std;
int main ()
{
  string str ("steve jobs is legend");
  string::iterator it;

  str.erase (str.begin()+ 5, str.end()-7);

  cout << str << endl;
  return 0;
}
```

- A) jobs is
- B) steve legend
- C) steve
- D) steve jobs is

Answer: B) steve legend

Explanation:

- `str.begin() + 5` points to index 5 (space after "steve")
- `str.end() - 7` points to index 14 ('g' in "legend")
- `str.erase()` removes characters from index 5 to 13 (" jobs is ")
- Result: "steve" + "legend" = "steve legend"

Question 15: String Appending Methods

Which method do we use to append more than one character at a time?

- A) append()
- B) operator+=
- C) data()
- D) both append & operator+=

Answer: D) both append & operator+=

Explanation:

- ✅ `append()` method can append strings or substrings
- ✅ `operator+=` can also append strings
- ❌ `data()` returns a pointer to internal character array, not for appending

---

Question 16: Character Array Reversal Error

What is the output of this code?

```cpp
int main()
{
    char p[20];
    char s[] = "string";
    int length = strlen(s);
    int i;

    for (i = 0; i < length; i++)
        p[i] = s[length-i];  // Bug: should be s[length-i-1]

    cout << p << endl;
}
```

- A) gnirts
- B) gnirt
- C) string
- D) no output is printed

Answer: D) no output is printed

Explanation: The loop assigns `p[0] = s[6]` which is '\0' (null terminator). Since the first character is null, `cout << p` prints nothing.

Question 17: Pointer Arithmetic with Strings

What should be '?' if we want to print 'Quiz'?

```cpp
```

```
int main()
{
   char arr[] = "HelloQuiz";
   cout << ? << endl;
   return 0;
}
```

- A) arr
- B) arr+5
- C) arr+4
- D) Not possible

Answer: B) arr+5

Explanation: The string "HelloQuiz" has 'Q' at index 5. `arr + 5` points to the memory location starting from 'Q', so `cout << arr + 5` prints "Quiz".

Question 18: Multiple String Access Methods

What will be the output?

cpp

```
int main()
{
   char str[] = "HelloQuiz";
   cout << &str[5] << " " << str+5 <<" ";
   cout << str[5] << " " << 5[str] << " " << endl;
   return 0;
}
```

- A) Quiz Quiz Q Q
- B) Quiz Quiz u u
- C) Quiz uiz u u
- D) None

Answer: A) Quiz Quiz Q Q

Explanation:

- `&str[5]` and `str+5` both point to index 5 ('Q'), printing "Quiz Quiz"

- `str[5]` and `5[str]` both access the character at index 5, printing "Q Q"

Question 19: Array Size vs String Length

What will be the output?

```cpp
int main()
{
    char str1[] = "HelloQuiz";
    char str2[] = {'H', 'e', 'l', 'l', 'o', 'Q', 'u', 'i', 'z'};

    int n1 = sizeof(str1)/sizeof(str1[0]);
    int n2 = sizeof(str2)/sizeof(str2[0]);

    cout << n1 << " " << n2 << endl;
    return 0;
}
```

- A) 10 9
- B) 10 10
- C) 9 10
- D) 9 9

Answer: A) 10 9

Explanation:

- `str1[]` includes automatic null terminator: 9 chars + 1 null = 10
- `str2[]` is explicit array with no automatic null terminator: exactly 9 chars

---

Question 20: String Concatenation Memory Issue

Which change can fix the segmentation fault in this concatenation function?

```cpp
```

```
void myStrcat(char a[], char b[])
{
   int i;
   int m = strlen(a);
   int n = strlen(b);
   for (i=0; i <= n; i++)
      a[m+i] = b[i];
}

int main()
{
   char str1[] = "Hello";  // Only 6 bytes
   char str2[] = "Quiz";
   myStrcat(str1, str2);
   cout << str1 << endl;
   return 0;
}
```

- A) char str1[] = "Hello"; → char str1[100] = "Hello";
- B) Above change + add a[m+n-1] = '\0'
- C) Only add a[m+n-1] = '\0'
- D) None

Answer: A) char str1[] = "Hello"; → char str1[100] = "Hello";

Explanation: The segmentation fault occurs because `str1[]` only has 6 bytes but we're trying to append 5 more characters. Allocating sufficient space fixes the issue. The null terminator is already copied by the loop.

Question 21: Remove Consecutive Duplicates

What is the output?

```
cpp
```

```
int fun(char p[])
{
   int current = 1, i = 1;
   while (p[current])
   {
      if (p[current] != p[current-1])
      {
         p[i] = p[current];
         i++;
      }
      current++;
   }
   p[i]='X';
   return i;
}

int main()
{
   char str[] = "babbar";
   int n = fun(str);
   for(int i=0; i<n; i++){
      cout << str[i];
   }
   return 0;
}
```

Answer: babar

Explanation: The function removes consecutive duplicate characters:

- "babbar" → keeps 'b', 'a', 'b' (different from prev 'a'), skips 'b' (same as prev), keeps 'a', 'r'
- Result: "babar", function returns 5, prints first 5 characters

---

Question 22: String Reversal with Bounds Error

What is the output?

```
cpp
```

```cpp
int main() {
    char p[] = "codewithbabbar";
    char t;
    int i, j;

    for(i=0, j=strlen(p); i < j; i++, j--)
    {
        t = p[i];
        p[i] = p[j-i];  // Bug: should be p[j-1]
        p[j-1] = t;
    }

    cout << p << endl;
    return 0;
}
```

- A) rabbabhtiwedoc
- B) codewithbabbar
- C) Nothing is printed on the screen/Error

Answer: C) Nothing is printed on the screen/Error

Explanation: The code has a bounds error: `p[i] = p[j-i]` tries to access `p[15-0] = p[15]` which is out of bounds (valid indices are 0-14). This causes undefined behavior or runtime error.

Question 23: Array Size

What is the output? (Assume char takes 1 byte)

```cpp
int main()
{
    char str[20] = "codewithbabbar";
    cout << sizeof(str) << endl;
    return 0;
}
```

- A) 10
- B) 20
- C) 40
- D) 80

Answer: B) 20

Explanation: `sizeof(str)` returns the size of the entire array (20 bytes), not the length of the string inside it. The string "codewithbabbar" is 14 characters, but the array size is fixed at 20.

Question 24: String Concatenation with strcat

What is the output?

cpp

```cpp
int main()
{
   char s1[50];
   char s2[50];

   strcpy(s1, "code");
   strcpy(s2, "help");
   strcat(s1, s2);

   cout << s1 << endl;
   return 0;
}
```

- A) None
- B) codehelp
- C) helpcode
- D) plehedoc

Answer: B) codehelp

Explanation:

- `strcpy(s1, "code")` copies "code" to s1
- `strcpy(s2, "help")` copies "help" to s2
- `strcat(s1, s2)` appends s2 to s1, making s1 = "codehelp"

Question 25: Character Removal Function

What does this code do?

cpp

```
void fun(char input[], char c) {
    int i = 0, j = 0;
    while (input[i] != '\0') {
        if (input[i] != c) {
            input[j] = input[i];
            ++j;
        }
        i++;
    }
    input[j] = '\0';
}
```

- A) Remove all occurrences of target c
- B) Add c to the string
- C) Reverse the string
- D) None

Answer: A) Remove all occurrences of target c

Explanation: The function iterates through the input string, copying only characters that are not equal to `c` to a new position. It effectively removes all occurrences of character `c` from the string.

Question 26: Most Frequent Character

What does this code do?

```cpp
```

```
char fun(char input[]) {
   char c;
   int count = 0;

   for (int i = 0; input[i] != '\0'; i++) {
      int sum = 0;

      for (int j = 0; input[j] != '\0'; j++) {
         if (input[i] == input[j]) {
            sum = sum + 1;
         }
      }

      if (sum > count) {
         count = sum;
         c = input[i];
      }
   }

   return c;
}
```

- A) Returns element with even frequency
- B) Returns element with odd frequency
- C) Returns element with lowest frequency
- D) Returns element with highest frequency

Answer: D) Returns element with highest frequency

Explanation: The function counts the frequency of each character and keeps track of the character with the maximum frequency. It returns the most frequently occurring character.

Question 27: Palindrome Check

Is '(())' a palindrome?

- A) Yes
- B) No

Answer: A) Yes

Explanation: A palindrome reads the same forwards and backwards. "(())" when reversed is still "(())", so it is a palindrome.

Question 28: Character Replacement Function

What does this code do?

```cpp
void fun(char input[], char c1, char c2) {
    for (int i = 0; input[i] != '\0'; i++) {
        if (input[i] == c1)
            input[i] = c2;
    }
}
```

- A) It counts the occurrence of c1
- B) It counts the occurrence of c2
- C) It replaces c1 with c2
- D) It replaces c2 with c1

Answer: C) It replaces c1 with c2

Explanation: The function loops through the input string and replaces every occurrence of character `c1` with character `c2`.

Summary

This collection covers essential C++ string and character manipulation concepts including:

- String member functions (empty, append, assign, erase)
- Character array vs std::string differences
- String concatenation methods
- Pointer arithmetic with strings
- C-style string functions (strcpy, strcat, strlen)
- String iterators and algorithms
- Memory management issues
- Common string manipulation patterns

Each question tests practical understanding of C++ string handling, memory management, and common pitfalls in string operations.